

Connectivity and Recursive / Depth-First Search

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Simple Connectivity Questions

- Some of the most fundamental graph algorithms related to issues of connectivity:
 - Are nodes i and j connected by some path?
 - If so, determine such a path.
 - In a digraph, is there a directed path from i to j ?
 - Does a (directed) graph have a (directed) cycle?
 - Partition a graph into its connected components.
 - Partition a digraph into its strongly connected components.
- We can answer all of these questions in linear time using a remarkably versatile algorithm known as **depth-first search**.

Simple Connectivity Questions

- Some of the most fundamental graph algorithms related to issues of connectivity:
 - Are nodes i and j connected by some path?
 - If so, determine such a path.
 - In a digraph, is there a directed path from i to j ?
 - Does a (directed) graph have a (directed) cycle?
 - Partition a graph into its connected components.
 - Partition a digraph into its strongly connected components.
- We can answer all of these questions in linear time using a remarkably versatile algorithm known as **depth-first search**.

Depth-First Search (DFS)

- Straightforward recursive search / floodfill.
- When visiting node i , mark it as having been visited, then recursively visit all unvisited neighbors.

Depth-First Search (DFS)

- `DFS-Visit(i) :`
 `Status[i] = 'visited'`
 For all `j` such that `(i,j)` is an edge:
 If `Status[j] = 'unvisited' : DFS-Visit(j)`
- `Full-DFS :`
 For all `i`: `Status[i] = 'unvisited'`
 For all `i`: If `Status[i] = 'unvisited' : DFS-Visit(i)`
- Works in undirected and directed graphs.
- Full-DFS takes $O(m + n)$ time since it spends $O(1)$ time on each node and edge (convention: $n = \#$ nodes, $m = \#$ edges)
 - You might sometimes see this written as $O(|V| + |E|)$, where V and E denote the sets of vertices and edges in our graph.
 - You might also see it as $O(m)$ or $O(|E|)$, assuming that term is dominant.

Depth-First Search (DFS)

- Full-DFS gives us an easy way to partition an undirected graph into connected components.
- If we mark nodes in each component with different labels, we can check in $O(1)$ time if node j is reachable from node i in an undirected graph.
 - In a directed graph, we'd rather launch a DFS at node i and see if it encounters j , to see if a directed $i \rightarrow j$ path exists.
- Can often augment DFS to return other useful information (e.g., component size).
- Let's work through a quick coding example...